

Tecnológico Nacional de México

Instituto Tecnológico de Chiná

Nombre de la Carrera:

INGENIERÍA INFORMÁTICA

Nombre de la Asignatura:

PROGRAMACION CLIENTE SERVIDOR

Semestre y grupo:

7AI

Actividad:

INVESTIGACIÓN U2

Nombre del estudiante:

MIGUEL EDUARDO CHI CHABLE

Nombre del Profesor:

ING.JENNER NOEL CHE MÉNDEZ

Lugar y Fecha:

CHINÁ, CAMPECHE, 31 DE OCTUBRE DEL 2026

Tabla de contenido

Introducción 3

2.2 Diseño API first con OpenAPI 3.1 o superior compatible 7

2.3 Validación de entrada manejo uniforme de errores serialización paginación filtrado ordenamiento y búsqueda 8

2.4 Acceso a MySQL MariaDB o PostgreSQL Supabase 10

Restricciones e integridad 11

2.5 Separación por capas 12

2.6 Consumo desde HTML Bootstrap JavaScript o Flutter Dart 13

2.7 Pruebas de endpoints y colección reproducible de solicitudes 15

Conclusión 16

Fuentes de consulta 17

Introducción

Una API (interfaz de programación de aplicaciones) permite que distintos programas intercambien información mediante reglas definidas. Por ejemplo, una aplicación web y una aplicación móvil pueden utilizar la misma API para consultar productos, registrar usuarios o realizar pedidos.

Desarrollar una API requiere definir cómo se identifican los recursos, cómo se reciben y validan los datos, cómo se almacenan y cómo se comunican los resultados al cliente.

2.1 Recursos URI métodos HTTP encabezados códigos de estado JSON idempotencia y restricciones REST

Recursos y URI

Un recurso es una entidad o concepto que la API permite consultar o modificar, como un producto, un cliente o un pedido. Su representación contiene la información que se intercambia con el cliente.

Una URI (identificador uniforme de recursos) identifica un recurso. Por ejemplo:

https://api.ejemplo.com/v1/productos/25

En este caso, la dirección identifica el producto con el número 25. La ruta /productos representa una colección y /productos/25, un elemento específico. Utilizar sustantivos para nombrar estas rutas facilita su comprensión. La sintaxis general de las URI está definida en el RFC 3986.

Métodos HTTP

Los métodos expresan la operación solicitada:

Método	Propósito habitual	Ejemplo
GET	Consultar una representación.	Obtener un producto.
POST	Procesar datos; frecuentemente, crear un recurso.	Registrar un pedido.
PUT	Crear o reemplazar el estado del recurso indicado.	Reemplazar los datos de un producto.
PATCH	Aplicar modificaciones parciales.	Cambiar su precio.
DELETE	Solicitar la eliminación del recurso.	Eliminar un producto.
HEAD	Obtener encabezados sin el cuerpo de una respuesta GET.	Comprobar metadatos.
OPTIONS	Consultar opciones de comunicación.	Conocer métodos permitidos.

Encabezados

Los encabezados transportan metadatos:

- Content-Type: formato del cuerpo enviado, como application/json.
- Accept: formatos de respuesta aceptados.
- Authorization: credenciales de autenticación.
- Location: dirección del recurso creado.

- Cache-Control: reglas de almacenamiento en caché.

Códigos de estado

Código	Significado
200 OK	Operación completada correctamente.
201 Created	Recurso creado.
204 No Content	Operación completada sin cuerpo de respuesta.
400 Bad Request	Solicitud incorrecta.
401 Unauthorized	Faltan credenciales válidas.
403 Forbidden	Acceso rechazado.
404 Not Found	Recurso no encontrado.
409 Conflict	Conflicto con el estado actual.
422 Unprocessable Content	Contenido comprendido, pero no procesable.
500 Internal Server Error	Error inesperado del servidor.

Idempotencia

Una operación es idempotente cuando repetir una solicitud idéntica produce el mismo efecto solicitado que ejecutarla una vez. GET, PUT y DELETE son idempotentes; POST y PATCH no lo garantizan.

Por ejemplo, eliminar dos veces un producto puede devolver códigos diferentes, pero el efecto final sigue siendo que el producto está eliminado. Estas reglas forman parte de la semántica HTTP.

JSON

JSON (JavaScript Object Notation) es un formato de texto para intercambiar datos estructurados. Admite objetos, arreglos, cadenas, números, booleanos y null.

```
{
  "id": 25,
  "nombre": "Memoria USB",
  "precio": 150.50,
  "disponible": true
```

```
}
```

Los nombres de propiedades y las cadenas utilizan comillas dobles. JSON estándar no admite comentarios ni comas finales. Aunque surgió de JavaScript, es independiente del lenguaje de programación.

Restricciones REST

REST (Representational State Transfer) es un estilo arquitectónico definido mediante restricciones:

1. Cliente-servidor: separa la interfaz del usuario de las responsabilidades del servidor.
2. Sin estado de sesión en el servidor: cada solicitud incluye el contexto necesario para procesarla. Esto permite almacenar recursos en una base de datos.
3. Caché: las respuestas indican si pueden reutilizarse.
4. Interfaz uniforme: comprende identificar recursos, manipularlos mediante representaciones, utilizar mensajes autodescriptivos y proporcionar enlaces que orienten las acciones disponibles.
5. Sistema por capas: permite intermediarios, como proxies y balanceadores.
6. Código bajo demanda, opcional: el servidor puede proporcionar código ejecutable al cliente.

Por tanto, utilizar HTTP y JSON no basta por sí solo para cumplir todas las restricciones REST.

2.2 Diseño API first con OpenAPI 3.1 o superior compatible

El enfoque API-first consiste en acordar el contrato de la API antes de desarrollar su implementación. El equipo define operaciones, datos, errores y ejemplos para que quienes construyen el servidor y los clientes trabajen con las mismas reglas.

OpenAPI permite expresar ese contrato en YAML o JSON. Su versión 3.1 integra el modelo de esquemas de JSON Schema 2020-12, lo que permite describir tipos y restricciones de datos. Para este ejemplo se utiliza OpenAPI 3.1.2.

Los elementos principales son:

Elemento	Función
info	Nombre, descripción y versión de la API.
servers	Direcciones de los servidores.
paths	Rutas y operaciones disponibles.
parameters	Datos recibidos en ruta, consulta, encabezados o cookies.
requestBody	Estructura del cuerpo de una solicitud.
responses	Respuestas y códigos posibles.
components.schemas	Modelos reutilizables.
examples	Casos concretos de uso.

Los esquemas establecen propiedades, tipos, campos obligatorios y límites. Los parámetros describen entradas como el identificador de un producto. Las respuestas especifican qué devuelve cada operación, y los ejemplos deben ser coherentes con el esquema.

Versionado

Deben distinguirse tres conceptos: openapi identifica la versión de la especificación; info.version, la versión del contrato; y /v1, una estrategia elegida para exponer la versión de la API.

Un cambio incompatible, como eliminar un campo obligatorio para los clientes, requiere una transición planificada. Puede publicarse una versión nueva y mantener temporalmente la anterior. También existen estrategias de versionado mediante parámetros o encabezados; lo esencial es documentar una política consistente.

2.3 Validación de entrada manejo uniforme de errores serialización paginación filtrado ordenamiento y búsqueda

Validación de entrada

La validación comprueba que los datos cumplen los requisitos antes de utilizarlos. Debe ejecutarse en el servidor, aunque también exista en el formulario del cliente.

Incluye dos niveles:

- Sintáctico: tipo, formato, longitud y estructura.
- Semántico: sentido del dato dentro de la operación.

Por ejemplo, un precio debe ser numérico y no negativo; la fecha final de una promoción debe ser posterior a la inicial. También deben limitarse el tamaño de las solicitudes y la cantidad de elementos recibidos.

Validar un identificador no demuestra que el usuario tenga permiso para-acceder al recurso: la autorización requiere una comprobación adicional.

Manejo uniforme de errores

Todos los endpoints deben comunicar los errores con una estructura predecible. El RFC 9457 define Problem Details, que puede enviarse como application/problem+json.

```
{
  "type": "https://api.ejemplo.com/problemas/validacion",
  "title": "Datos inválidos",
  "status": 422,
  "detail": "Uno o más campos incumplen las reglas.",
  "instance": "/v1/productos",
  "errors": [
    {
      "field": "precio",
      "message": "Debe ser mayor o igual que cero."
    }
  ]
}
```

Aquí, errors es una extensión propuesta para señalar los campos afectados. La respuesta pública debe explicar el problema sin revelar contraseñas, consultas SQL ni trazas internas.

Serialización

La serialización transforma objetos internos en un formato transmisible, como JSON; la deserialización realiza el proceso inverso.

Conviene definir modelos de entrada y salida para controlar los campos publicados. Por ejemplo, una respuesta de usuario puede incluir nombre y correo, mientras excluye el hash de contraseña. También deben acordarse las representaciones de fechas, valores nulos y cantidades monetarias. Los DTO —objetos de transferencia de datos— ayudan a mantener estos modelos separados de las entidades internas.

Operaciones sobre colecciones

Operación	Propósito	Ejemplo propuesto
Paginación	Dividir resultados en bloques.	?page=2&limit=20
Filtrado	Seleccionar registros por condiciones.	?categoria=accesorios
Ordenamiento	Definir la secuencia de resultados.	?sort=precio&order=asc
Búsqueda	Encontrar coincidencias de texto.	?q=memoria

Estas convenciones deben documentarse; no existe una sintaxis universal obligatoria para esos parámetros. La paginación puede utilizar desplazamientos o cursores. Un orden estable, por ejemplo por precio y después por identificador, ayuda a resolver empates.

Además, el servidor debe limitar el número de resultados y permitir únicamente campos de ordenamiento definidos por la aplicación. Los valores se pasan mediante consultas parametrizadas; los nombres de columnas y direcciones de orden se seleccionan desde opciones permitidas.

2.4 Acceso a MySQL MariaDB o PostgreSQL Supabase

La capa de persistencia conserva los datos y aplica reglas de integridad. Para ilustrar los conceptos se utiliza PostgreSQL; Supabase proporciona una plataforma basada en este motor.

Repositorios

Un repositorio encapsula operaciones de acceso a datos, por ejemplo:

```
ProductoRepositorio.buscarPorId(id)
ProductoRepositorio.listar(filtros)
ProductoRepositorio.guardar(producto)
```

Los servicios utilizan estas operaciones sin necesitar conocer cada consulta SQL. Este patrón facilita sustituir implementaciones y probar la lógica de negocio de forma aislada.

Las consultas deben separar instrucciones y valores:

```
SELECT id, nombre, precio
FROM productos
WHERE id = $1;
```

El valor de $1 se proporciona mediante el controlador de PostgreSQL. La parametrización evita que un dato recibido se interprete como parte de la sintaxis SQL.

Transacciones

Una transacción agrupa operaciones que deben completarse juntas. Al registrar un pedido, pueden necesitarse tres cambios: crear el pedido, insertar sus detalles y descontar existencias.

Si ocurre un error, se ejecuta ROLLBACK; si todo termina correctamente, se confirma con COMMIT. Para evitar ventas por encima del inventario disponible también debe controlarse la concurrencia, mediante actualizaciones condicionadas, bloqueos o un aislamiento adecuado.

Migraciones

Las migraciones son cambios versionados del esquema, como crear tablas, agregar columnas o establecer índices. Permiten reproducir la evolución de la base de datos en desarrollo, pruebas y producción.

Cada migración debe revisarse antes de aplicarse, especialmente cuando modifica datos existentes o elimina información.

Restricciones e integridad

Restricción	Función
PRIMARY KEY	Identificar cada fila.
FOREIGN KEY	Mantener relaciones válidas.
NOT NULL	Exigir un valor.
UNIQUE	Impedir duplicados en las columnas indicadas.
CHECK	Comprobar una condición.

Estas restricciones protegen los datos incluso cuando se escriben desde procesos diferentes de la API.

En Supabase, cuando los clientes acceden a tablas mediante su API, deben configurarse permisos y políticas de Row Level Security (RLS) para controlar qué filas puede consultar o modificar cada usuario.

2.5 Separación por capas

La separación por capas asigna responsabilidades específicas:

Capa	Responsabilidad	Ejemplo
Rutas y controladores	Recibir HTTP, interpretar entradas y producir respuestas.	Atender POST /pedidos.
Casos de uso o servicios	Coordinar una operación de negocio.	Registrar un pedido completo.
Dominio	Representar entidades y reglas del negocio.	Impedir pedidos sin productos.
Acceso a datos	Ejecutar consultas y persistir información.	Guardar el pedido en PostgreSQL.

La separación debe reflejarse en las dependencias del código. En una arquitectura con inversión de dependencias, el núcleo define interfaces y la infraestructura las implementa. Así, las reglas del negocio pueden probarse sin depender del servidor HTTP ni de una base de datos real.

Ventajas y limitaciones

La separación facilita localizar responsabilidades, reutilizar operaciones y sustituir componentes para pruebas. Sin embargo, cambiar de base de datos todavía puede requerir adaptar consultas, migraciones y tipos de datos; la arquitectura reduce el alcance del cambio, pero no elimina el trabajo.

También conviene distinguir capas lógicas de niveles físicos de despliegue. Separar responsabilidades en el código no obliga a ejecutarlas en máquinas diferentes. Distribuirlas físicamente puede mejorar ciertas capacidades de escalamiento, pero agrega comunicación de red y complejidad.

Para aplicar este enfoque, deben evitarse estos errores:

- Colocar toda la operación en el controlador.
- Escribir consultas SQL en varios componentes sin una responsabilidad clara.
- Depender del formulario para hacer cumplir las reglas del negocio.
- Crear servicios que solo trasladan datos sin aportar una separación útil.

2.6 Consumo desde HTML Bootstrap JavaScript o Flutter Dart

El cliente envía solicitudes, interpreta las respuestas y presenta información al usuario.En una aplicación web, HTML define la estructura, Bootstrap facilita el diseño visual y JavaScript realiza las solicitudes y actualiza la interfaz. Mediante fetch, JavaScript puede consultar la API.

Es necesario comprobar respuesta.ok: una respuesta HTTP como 404 o 500 no provoca automáticamente el rechazo de la promesa de fetch.

En Flutter/Dart, puede utilizarse el paquete http para enviar solicitudes, jsonDecode para interpretar JSON y modelos Dart para representar los datos. FutureBuilder permite construir la interfaz según el estado de una operación asíncrona.

Estados de la interfaz

Para el ejemplo de productos se propone distinguir:

Estado	Situación	Presentación
Carga	La solicitud sigue pendiente.	Indicador de progreso.
Error	Falló la conexión o la operación.	Mensaje y opción de reintentar.
Vacío	La consulta fue correcta, sin resultados.	No se encontraron productos.
Éxito	Se recibieron datos utilizables.	Lista o tabla de productos.

Distinguir vacío y error evita confundir una búsqueda sin coincidencias con una falla del sistema. Durante los envíos también conviene controlar los botones para reducir acciones duplicadas.

2. Proceso de comunicación

Para el ejemplo de una tienda, consultar productos implica:

1. El usuario abre la pantalla o presiona “Consultar”.
2. La aplicación muestra un indicador de carga.
3. Envía una solicitud GET al endpoint de productos.
4. Comprueba el código HTTP recibido.
5. Interpreta el cuerpo JSON.

CORS y diferencias entre clientes

Cuando una aplicación web consulta una API con un origen diferente —por dominio, protocolo o puerto— el navegador aplica las reglas de CORS. El servidor debe autorizar los orígenes, métodos y encabezados pertinentes.

Esto también afecta a Flutter cuando se ejecuta como aplicación web. Las aplicaciones móviles nativas no están sujetas al mecanismo CORS del navegador. En cualquier caso, la API necesita autenticar y autorizar las operaciones: CORS no sustituye esos controles.

Buenas prácticas para el ejemplo

Al implementar el cliente de productos, conviene:

- Mantener la dirección base de la API en una configuración.
- Separar las solicitudes de la presentación visual.
- Comprobar códigos HTTP y estructura de las respuestas.
- Establecer tiempos de espera y permitir reintentos controlados.
- Conservar los datos del formulario cuando ocurre un error.
- Evitar que una búsqueda anterior sobrescriba resultados más recientes.
- Probar los cuatro estados con respuestas controladas.

2.7 Pruebas de endpoints y colección reproducible de solicitudes

Las pruebas verifican que cada operación cumple su contrato: método, ruta, autorización, estructura de entrada, código de estado y respuesta.

Para la API del ejemplo, una matriz de pruebas sería:

Caso	Resultado esperado
Consultar un producto existente.	200 y datos conformes al esquema.
Consultar un identificador inexistente.	404.
Crear un producto válido.	201 y recurso creado.
Enviar un precio negativo.	422, según la política documentada.
Repetir un SKU único.	409, según el contrato.
Solicitar una operación protegida sin credenciales.	401.
Acceder a datos de otro usuario sin permiso.	Denegación conforme a la política.
Provocar un fallo durante un pedido.	Ausencia de escrituras parciales.

También deben probarse entradas demasiado grandes, formatos de contenido no admitidos y transiciones de negocio inválidas. Las comprobaciones deben ejecutarse en el servidor, independientemente de las restricciones del cliente.

Colección reproducible

Una colección reúne solicitudes guardadas con sus métodos, rutas, encabezados, cuerpos y verificaciones. En Postman, el Collection Runner permite ejecutarlas en un orden definido y utilizar scripts para transferir datos entre solicitudes.

Para que otra persona pueda repetir el proceso, la colección debe incluir:

1. Variables como base_url, token y producto_id.

2. Datos de prueba conocidos.

3. Una secuencia de preparación, ejecución y limpieza.

Conclusión

El desarrollo de una API confiable combina un contrato claro, el uso correcto de HTTP, validación del lado del servidor, protección de los datos y una distribución precisa de responsabilidades. OpenAPI facilita el acuerdo entre equipos; las transacciones y restricciones mantienen la integridad; y las pruebas reproducibles permiten comprobar que el comportamiento implementado coincide con lo documentado. En conjunto, estos elementos facilitan que aplicaciones web y móviles consuman el mismo servicio de forma consistente.

Fuentes de consulta

Cómo funciona HTTP: métodos, cabeceras y códigos de estado. (2026, septiembre 17). Escueladeprogramacion.net. https://escueladeprogramacion.net/blog/como-funciona-http-metodos-cabeceras-y-codigos-de-estado

(S/f). Scribd.com. Recuperado el 31 de octubre de 2026, de https://es.scribd.com/document/873731968/2-1-1-Fundamentos-de-HTTP-y-REST-Conceptos-y-Buenas-Practicas-2

EDICOM - Portal de empleo de EDICOMGroup. (2026, enero 9). *BACKEND. Buenas prácticas al desarrollar una API REST*. EDICOM Careers. https://careers.edicomgroup.com/blogtech/backend-buenas-practicas-al-desarrollar-una-api-rest/

OpenAPI Specification v3.1.1. (s/f). Openapis.org. Recuperado el 31 de octubre de 2026, de https://spec.openapis.org/oas/v3.1.1.html

Peiro, E. (2026, abril 24). Cómo crear una API RESTful: Principios, diseño y mejores prácticas. *Aprender21*. https://www.aprender21.mx/blog/como-crear-api-restful-principios-ejemplos

(S/f-b). Github.com. Recuperado el 31 de octubre de 2026, de https://github.com/Ezze191/aplicaciones-web-proyecto/blob/main/README.md

Manejar todos los Errores y Excepciones. (s/f). Owasp.org. Recuperado el 31 de octubre de 2026, de https://devguide.owasp.org/es/04-design/02-web-app-checklist/10-handle-errors-exceptions/

de Errores y Depuración, M. (s/f). *Unidad 10 Manejo de Errores y Depuración*. Com.mx. Recuperado el 31 de octubre de 2026, de https://iitec.appsiapi.com.mx/cargas/pdf/69c0596bd4c0e85cfbb4db29407d7a84_nsk.pdf

Salgado, L. G. (2023, noviembre 28). Técnicas de validación de entradas en programación. *KeepCoding Bootcamps*. https://keepcoding.io/blog/validacion-de-entradas-en-programacion/

(S/f). Redhat.com. Recuperado el 31 de octubre de 2026, de https://docs.redhat.com/es/documentation/red_hat_enterprise_linux/8/html/deploying_different_types_of_servers/using-mariadb

Desarrolladoresweb.org, P. (2024, noviembre 30). *Descubre PostgreSQL: Qué es, instalación, características y todo lo que necesitas para empezar hoy*. Web Devs. https://desarrolladoresweb.org/postgresql/postgresql-guia-completa-sobre/

Modelo OSI: 7 capas y ciberataques habituales. (2023, abril 27). *Stackscale*. https://www.stackscale.com/es/blog/modelo-osi/

Jimenez, A. (2024, noviembre 16). *Estados de carga y errores en Flutter*. Platzi INC. Platzi.com. https://platzi.com/cursos/flutter/estados-de-carga-y-errores-en-flutter/